

16

Getting Started with Next.js

Up until now, we have been using various libraries and tools to develop full-stack web applications. Now, we introduce Next.js as an enterprise-ready full-stack web application framework for React. Next.js combines all the functions and tools you need for full-stack web development in one package. In this book, we use Next.js because it is currently the most popular framework supporting all new React features, such as React Server Components and Server Actions, which are the future of full-stack React development. However, there are other frameworks for full-stack React, such as Remix, which have recently also started supporting the new React features.

In this chapter, we will learn how Next.js works and what its advantages are. Then, we will re-create our blog project in Next.js to highlight the differences between using a simple bundler such as Vite, and a full framework such as Next.js. Along the way, we will learn how the Next.js App Router works. Finally, we are going to re-create our (static) blog app by creating components and pages and then defining links between them.

In this chapter, we are going to cover the following main topics:

- What is Next.js?
- Setting up Next.js
- Introducing the App Router
- Creating static components and pages

Technical requirements

Before we start, please install all the requirements from [***Chapter 1***](#), *Preparing for Full-Stack Development*, and [***Chapter 2***](#), *Getting to Know Node.js and MongoDB*.

The versions listed in those chapters are the ones used in the book. While installing a newer version should not be an issue, please note that certain steps might work differently on a newer version. If you are having an issue with the code and steps provided in this book, please try using the versions mentioned in *Chapters 1* and *2*.

You can find the code for this chapter on GitHub:

<https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch16>.

The CiA video for this chapter can be found at:

<https://youtu.be/jQFCZqCspoc>.

What is Next.js?

Next.js is a React framework that puts together everything you need to create a full-stack web application with React. Its main features are as follows:

- Good developer experience out of the box, including hot module reloading, error handling, and more.
- File-based routing and nested layouts, route handlers to define API endpoints, all from Next.js.
- **Internationalization (i18n)** support in routing, allowing us to create internationalized routes.
- Enhanced server-side and client-side data fetching with caching out of the box.
- Middleware to run code before requests are completed.
- Options to run API endpoints on serverless runtimes.
- Out-of-the-box support for static generation of pages.
- Dynamic streaming of components when they are needed, allowing us to show an initial page quickly, and then load other components later.
- Advanced client and server rendering, allowing us to not only render React components on the server side (**server-side rendering (SSR)**)

but also make use of **React Server Components**, which allow us to render React components exclusively on the server without sending additional JavaScript to the client.

- **Server Actions** to progressively enhance forms and actions sent from the client to the server, allowing us to submit forms even without JavaScript on the client.
- Built-in optimizations for images, fonts, and scripts to improve Core Web Vitals.
- Additionally, Next.js provides a platform to easily deploy our apps on – Vercel.

All in all, Next.js puts together everything we have learned about full-stack development throughout this book, refines each concept and makes it more advanced and customizable, and provides all of that in one single package. We are now going to re-create the blog application from earlier chapters, but from scratch with Next.js. Doing so will allow us to see the differences between developing an app with and without a full-stack framework.

Setting up Next.js

We are now going to set up a new project using the **create-next-app** tool, which sets up everything automatically for us. Follow these steps to get started:

1. Open a new Terminal window. Make sure you are outside of any project folders. Run the following command to create a new folder and initialize a Next.js project there:

```
$ npx create-next-app@14.1.0
```

2. When asked if it's **Ok to proceed?**, press **y** and confirm by pressing *Return/Enter*.
3. Give the project a name, such as **ch16**.
4. Answer the questions as follows:
 1. **Would you like to use TypeScript?: No**
 2. **Would you like to use ESLint?: Yes**
 3. **Would you like to use Tailwind CSS?: No**

4. Would you like to use ``src/`` directory?: Yes
 5. Would you like to use App Router?: Yes
 6. Would you like to customize the default import alias?: No
5. After answering all the questions, a new Next.js app will be created in the **ch16** folder. The output should look as follows:

```

➔ ~ /D/Full-Stack-React-Projects main > npx create-next-app@14.1.0
Need to install the following packages:
create-next-app@14.1.0
Ok to proceed? (y) y
✔ What is your project named? ... ch16
✔ Would you like to use TypeScript? ... No / Yes
✔ Would you like to use ESLint? ... No / Yes
✔ Would you like to use Tailwind CSS? ... No / Yes
✔ Would you like to use `src/` directory? ... No / Yes
✔ Would you like to use App Router? (recommended) ... No / Yes
✔ Would you like to customize the default import alias (@/*)? ... No / Yes
Creating a new Next.js app in /Users/dan/Development/Full-Stack-React-Projects/ch16.

Using npm.

Initializing project with template: app

Installing dependencies:
- react
- react-dom
- next

Installing devDependencies:
- eslint
- eslint-config-next

added 305 packages, and audited 306 packages in 3s

120 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
Success! Created ch16 at /Users/dan/Development/Full-Stack-React-Projects/ch16

```

Figure 16.1 – Creating a new Next.js project

6. Open the newly created **ch16** folder in VS Code.
7. In the new VS Code window, open a Terminal and run the project with the following command:

```
$ npm run dev
```

8. Open **http://localhost:3000** in your browser to see the Next.js app running! The app should look as follows:

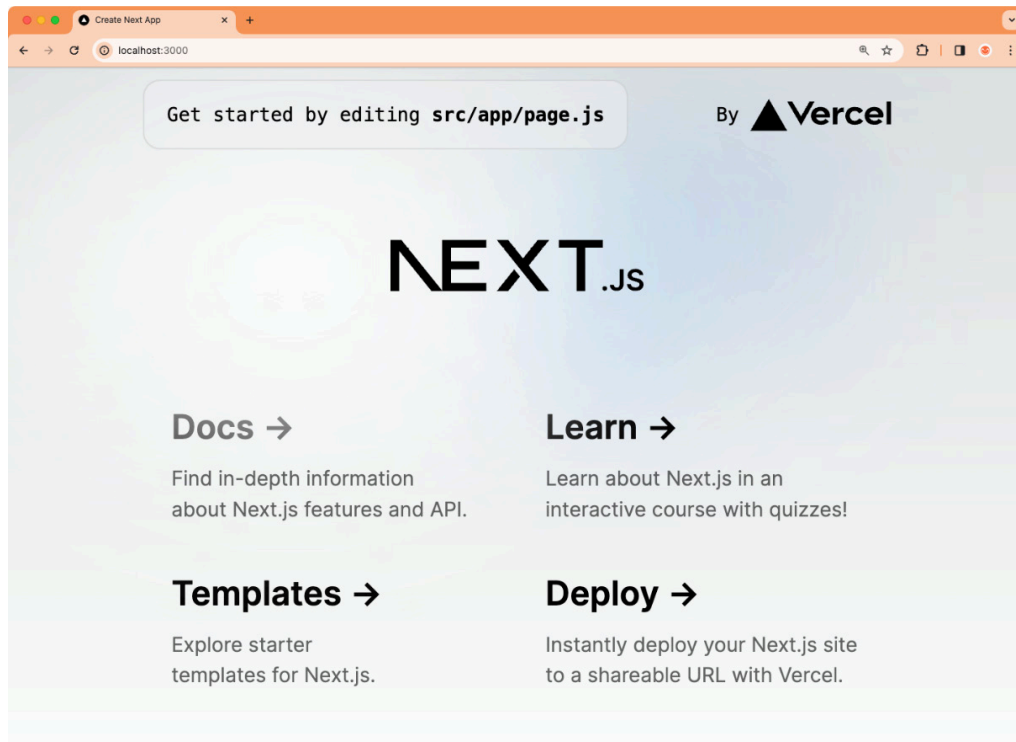


Figure 16.2 – Our newly created Next.js app running in the browser

9. Unfortunately, **create-next-app** does not set up Prettier for us, so let's quickly do that now. Install Prettier by running the following command:

```
$ npm install --save-dev prettier@2.8.4 \
  eslint-config-prettier@8.6.0
```

10. Create a new **.prettierrc.json** file in the root of the project, with the following contents:

```
{
  "trailingComma": "all",
  "tabWidth": 2,
  "printWidth": 80,
  "semi": false,
  "jsxSingleQuote": true,
  "singleQuote": true
}
```

11. Edit the existing **.eslintrc.json** to extend from **prettier**, as follows:

```
{
```

```
"extends": ["next/core-web-vitals", "prettier"]
}
```

12. Go to the VS Code workspace settings, change the **Editor: Default Formatter** setting to **Prettier**, and check the checkbox for **Editor: Format On Save**.

Now we have successfully created a new Next.js project with ESLint and Prettier! We could still set up Husky and lint-staged, just like we have done before, but for now, we are going to stick with this simple setup. Next, we are going to learn more about how apps are structured in Next.js.

Introducing the App Router

Next.js comes with a special paradigm for structuring applications called the App Router. The App Router makes use of the folder structure in the **src/app/** folder to create routes for our apps. The root folder (**/** path) is **src/app/**. If we want to define a path, such as **/posts**, we need to create a **src/app/posts/** folder. To make this folder a valid route, we need to put a **page.js** file inside it, which contains the page component that will be rendered when visiting that route.

NOTE

*Alternatively, we can put a **route.js** file into a folder to turn it into an API route instead of rendering a page. We are going to learn more about API routes in [Chapter 18](#), *Advanced Next.js Concepts and Optimizations*.*

Additionally, Next.js allows us to define a **layout.js** file, which will be used as the layout for a certain path. The layout component accepts children, which can contain other layouts or pages. This flexibility allows us to define nested routes with sub-layouts.

There are other special files in the App Router paradigm, such as the **error.js** file, which will be rendered when there is an error on the page, and the **loading.js** file, which will be rendered while the page is loading (using React Suspense).

Take a look at the following example of a folder structure with the App Router:

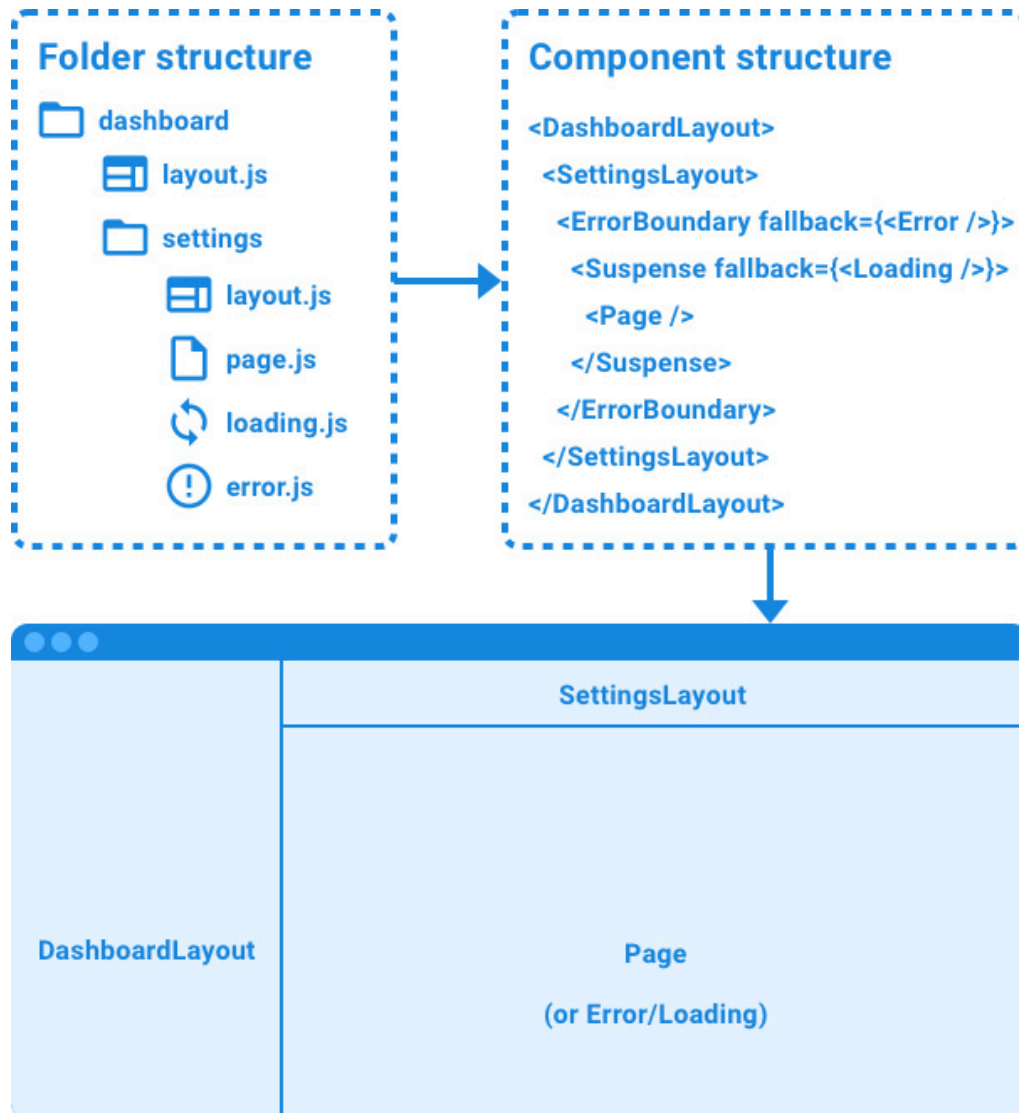


Figure 16.3 – Example of a folder structure with the App Router

In the preceding example, we have a `dashboard/settings/` route, defined by the `dashboard` and `settings` folders. The `dashboard` folder does not have a `page.js` file, so going to `dashboard/` will result in a **404 Not Found** error. However, the `dashboard` folder has a `layout.js` file, which defines the main layout of the dashboard. The `settings` folder has another `layout.js` file, which defines the layout of the settings page on the dashboard. It also has a `page.js` file, which will be rendered when the `dashboard/settings/` route is visited. Additionally, it has a `loading.js` file, which is rendered inside the settings layout, while the settings page is loading. It also contains an `error.js` file, which is rendered inside the settings layout if there is an error while loading the settings page.

As we can see, the App Router makes it easy to implement common use cases, such as nested routes, layouts, errors, and loading components. Let's now get started with defining the folder structure for our blog app.

Defining the folder structure

Let's recap and refine the routing structure of the blog application from previous chapters:

- `/` – the index page of our blog, containing a list of posts
- `/login` – the login page to login to an existing account
- `/signup` – the signup page to create a new account
- `/create` – a page to create a new blog post (this route is new)
- `/posts/:id` – a page to view a single blog post

All of these pages share a common layout with a navigation bar at the top, allowing us to navigate between the various pages of our app.

Let's now create this routing structure as a folder structure in the App Router:

1. *Delete* the existing `src/app/` folder.
2. Create a new `src/app/` folder. Inside it, create a `src/app/layout.js` file with the following contents:

```
export const metadata = {
  title: 'Full-Stack Next.js Blog',
  description: 'A blog about React and Next.js',
}
export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        <main>{children}</main>
      </body>
    </html>
  )
}
```

The `metadata` object is a special exported object in Next.js used to provide meta tags, such as the `<title>` and `<meta name="description">`

tags.

The default export of files in the App Router needs to be the component that should be rendered for the respective layout/page.

3. Create a new **src/app/page.js** file, with the following placeholder contents:

```
export default function HomePage() {  
  return <strong>Blog home page</strong>  
}
```

4. Create a new **src/app/login/** folder. Inside it, create a **src/app/login/page.js** file with the following placeholder contents:

```
export default function LoginPage() {  
  return <strong>Login</strong>  
}
```

5. Create a new **src/app/signup/** folder. Inside it, create a **src/app/signup/page.js** file with the following placeholder contents:

```
export default function SignupPage() {  
  return <strong>Signup</strong>  
}
```

6. Create a new **src/app/create/** folder. Inside it, create a **src/app/create/page.js** file with the following placeholder contents:

```
export default function CreatePostPage() {  
  return <strong>CreatePost</strong>  
}
```

7. Create a new **src/app/posts/** folder. Inside it, create a new **src/app/posts/[id]/** folder. This is a special folder containing a route parameter **id**, which we can use when rendering the page.
8. Create a new **src/app/posts/[id]/page.js** file with the following placeholder contents:

```
export default function ViewPostPage({ params }) {  
  return <strong>ViewPost {params.id}</strong>  
}
```

As you can see, we are getting the **id** from the **params** object provided by Next.js.

9. If it's not running anymore, start the Next.js dev server with the following command:

```
$ npm run dev
```

10. Then go to **http://localhost:3000/** (or refresh the page) in your browser to see the main route working. Go to the different routes, such as **/login** and **/posts/123**, to see the different pages being rendered and the **route** param working!

Now that we defined the folder structure for our project, let's continue by creating static components and pages.

Creating static components and pages

For the components in our blog, we can reuse a lot of the code we wrote in previous chapters, as it is not that much different in Next.js than it is in plain React. Only specific components, such as the navigation bar, will be different, because Next.js has its own router. We are going to create most of our components in a separate **src/components/** folder. This folder will only contain React components that can be reused across multiple pages. All page and layout components will still be in **src/app/**.

NOTE

In Next.js, it is also possible to co-locate regular components with the page and layout components, which should be done in large-scale projects for components that are only used on those specific pages. In small projects, it does not really matter as much, and we can just put all our regular components in a separate folder to make them easier to distinguish from page and layout components.

Defining components

Let's now get started with creating the components for our blog app:

1. Create a new **src/components/** folder.
2. Create a new **src/components/Login.jsx** file. Inside it, define a **<form>** with a **username** field, a **password** field, and a submit button:

```
export function Login() {
  return (
    <form>
      <div>
        <label htmlFor='username'>Username: </label>
        <input type='text' name='username' id='username' />
      </div>
      <br />
      <div>
        <label htmlFor='password'>Password: </label>
        <input type='password' name='password' id='password' />
      </div>
      <br />
      <input type='submit' value='Log In' />
    </form>
  )
}
```

NOTE

*We use uncontrolled input fields (so, no **useState** hooks) here on purpose, as it is not necessary to make controlled input fields for submitting forms with Server Actions, which we are going to learn about in the next chapter, [Chapter 17](#), *Introducing React Server Components*. However, it is important to properly define the **name** property of input fields, as that is what will be used to identify the field when the form is submitted.*

3. In a similar fashion, create a new **src/components/Signup.jsx** file and define a form with the same fields:

```
export function Signup() {
  return (
    <form>
      <div>
        <label htmlFor='username'>Username: </label>
        <input type='text' name='username' id='username' />
      </div>
```

```

    <br />
    <div>
      <label htmlFor='password'>Password: </label>
      <input type='password' name='password' id='password' />
    </div>
    <br />
    <input type='submit' value='Sign Up' />
  </form>
)
}

```

4. Create a new **src/components/CreatePost.jsx** file and define a form with a required **title** input field, a **textarea** to define the **contents**, and a submit button:

```

export function CreatePost() {
  return (
    <form>
      <div>
        <label htmlFor='title'>Title: </label>
        <input type='text' name='title' id='title' required />
      </div>
      <br />
      <textarea name='contents' id='contents' />
      <br />
      <br />
      <input type='submit' value='Create' />
    </form>
  )
}

```

5. Create a new **src/components/Post.jsx** file. As an improvement over the structure from previous chapters, the **Post** component will be used in the **PostList**, and only show the **title** and **author** of a blog post, with a link to the full post:

```

import PropTypes from 'prop-types'
export function Post({ _id, title, author }) {
  return (
    <article>
      <h3>{title}</h3>
      <em>
        Written by <strong>{author.username}</strong>

```

```

    </em>
  </article>
)
}

```

6. We also need to define **propTypes**. In this case, we will use a structure similar to the result from a database query, as we will be able to directly use database results when we introduce React Server Components in the next chapter:

```

Post.propTypes = {
  _id: PropTypes.string.isRequired,
  title: PropTypes.string.isRequired,
  author: PropTypes.shape({
    username: PropTypes.string.isRequired,
  }).isRequired,
  contents: PropTypes.string,
}

```

7. Create a new **src/components/PostList.jsx** file. Here, we are going to reuse the **propTypes** from the **Post** component, so let's also import the **Post** component:

```

import { Fragment } from 'react'
import PropTypes from 'prop-types'
import { Post } from './Post.jsx'

```

8. Then, we define the **PostList** component, which renders each blog post with the **Post** component:

```

export function PostList({ posts = [] }) {
  return (
    <div>
      {posts.map((post) => (
        <Fragment key={`post-${post._id}`}>
          <Post _id={post._id} title={post.title} author={post.author} />
          <hr />
        </Fragment>
      ))}
    </div>
  )
}

```

NOTE

*It is best practice to use a unique ID for the **key** prop, such as a database ID, so that React can keep track of items changing in a list.*

9. We now define the **propTypes** for the **PostList** component by making use of the existing **Post.propTypes**:

```
PostList.propTypes = {
  posts: PropTypes.arrayOf(
    PropTypes.shape(Post.propTypes)
  ).isRequired,
}
```

10. Lastly, we create a new **src/components/FullPost.jsx** file, in which we display the full post with all its contents:

```
import PropTypes from 'prop-types'
export function FullPost({ title, contents, author }) {
  return (
    <article>
      <h3>{title}</h3>
      <div>{contents}</div>
      <br />
      <em>
        Written by <strong>{author.username}</strong>
      </em>
    </article>
  )
}
```

11. Instead of reusing **propTypes** from the **Post** component, we are re-defining them here, because the **FullPost** component needs different props than the **Post** component (it does not have the **_id** prop, but instead has the **contents** prop):

```
FullPost.propTypes = {
  title: PropTypes.string.isRequired,
  author: PropTypes.shape({
    username: PropTypes.string.isRequired,
  }).isRequired,
```

```
contents: PropTypes.string,  
}
```

Now that we have defined all the components we are going to need for our blog app, let's move on to properly defining the page components.

Defining pages

After creating various components that we are going to need for our blog app, let's now replace the placeholder page components with proper pages that render the appropriate components. Follow these steps to get started:

1. Edit `src/app/login/page.js` and import the **Login** component, then render it:

```
import { Login } from '@components/Login'  
export default function LoginPage() {  
  return <Login />  
}
```

NOTE

Remember when we set up Next.js and were asked if we wanted to customize the default import alias? This import alias allows us to reference the `src/` folder of our project, making our imports absolute rather than relative. By default, this is done using the `@` alias. So, we can now just import from `@/components/Login` to import from the `src/components/Login.jsx` file, instead of having to import from `../../components/Login.jsx`. Absolute imports with import aliases become especially useful in large projects and make it easy to re-structure projects later.

2. Edit `src/app/signup/page.js` and, in a similar fashion, import and render the **Signup** component:

```
import { Signup } from '@components/Signup'  
export default function SignupPage() {  
  return <Signup />  
}
```

3. Repeat the process by editing the `src/app/create/page.js` file as follows:

```
import { CreatePost } from '@components/CreatePost'
export default function CreatePostPage() {
  return <CreatePost />
}
```

4. Now, edit the `src/app/posts/[id]/page.js` file and import the **FullPost** component:

```
import { FullPost } from '@components/FullPost'
```

5. Then, define a sample **post** object:

```
export default function ViewPostPage({ params }) {
  const post = {
    title: `Hello Next.js (${params.id})`,
    contents: 'This will be fetched from the database later',
    author: { username: 'Daniel Bugl' },
  }
}
```

To show that the param still works, we also put the **id** into the **title**.

6. Render the **FullPost** component, as follows:

```
return (
  <FullPost
    title={post.title}
    contents={post.contents}
    author={post.author}
  />
)
}
```

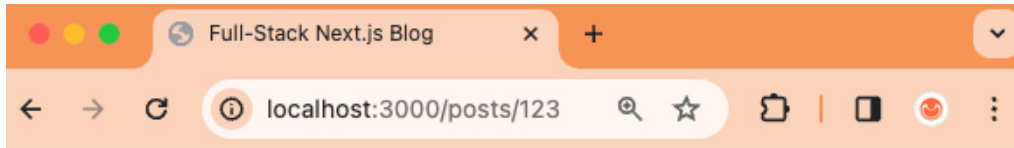
7. Lastly, edit `src/app/page.js` by importing the **PostList** component, creating an example **posts** array, and rendering the **PostList** component:

```
import { PostList } from '@components/PostList'
export default function HomePage() {
  const posts = [
    { _id: '123', title: 'Hello Next.js', author: { username: 'Daniel Bugl' } }
```



```
    ]  
    return <PostList posts={posts} />  
  }  
}
```

8. Go to `http://localhost:3000/posts/123` to see the **FullPost** component being rendered with the **id** param in the title. Feel free to change the **id** in the URL to see how the title changes. The following screenshot shows the **FullPost** component being rendered on the `/posts/123` path:



Hello Next.js (123)

This will be fetched from the database later

Written by Daniel Bugl

Figure 16.4 – Rendering the FullPost component with a Next.js route param in the title

After successfully defining all our pages, we still need a way to navigate between them, so let's continue by adding links between pages.

Adding links between pages

As mentioned earlier in this chapter, Next.js provides its own routing solution – the App Router. The routes are defined by the folder structure in the `src/app/` directory, and they all work already. All that's left to do now is to add links between them. To do this, we need to use the **Link** component from `next/link`. Follow these steps to get started with implementing a navigation bar:

1. Create a new `src/components/Navigation.jsx` file, where we import the **Link** component and **PropTypes**:

```
import Link from 'next/link'
import PropTypes from 'prop-types'
```

2. Define a **UserBar** component, which will be rendered when the user is logged in and allow a user to access the **Create Post** page and log out:

```
export function UserBar({ username }) {
  return (
    <form>
      <Link href='/create'>Create Post</Link> | Logged in as{' '}
      <strong>{username}</strong> <button>Logout</button>
    </form>
  )
}
UserBar.propTypes = {
  username: PropTypes.string.isRequired,
}
```

3. Then, define a **LoginSignupLinks** component, which will be rendered when the user is not logged in yet. It provides links to the **/login** and **/signup** pages to allow users to sign up and log in to our app:

```
export function LoginSignupLinks() {
  return (
    <div>
      <Link href='/login'>Log In</Link> | <Link href='/signup'>Sign Up</Link>
    </div>
  )
}
```

4. Next, define a **Navigation** component, which adds a link to the home page, and then conditionally renders either the **UserBar** component, or the **LoginSignupLinks** component, depending on whether the user is logged in or not:

```
export function Navigation({ username }) {
  return (
    <>
      <Link href='/'>Home</Link>
      {username ? <UserBar username={username} /> : <LoginSignupLinks />}
    </>
  )
}
```

```
}  
Navigation.propTypes = {  
  username: PropTypes.string,  
}
```

5. Now we just need to render the **Navigation** component. To make sure it appears on all pages of our blog app, we are going to put it in the root layout. Edit **src/app/layout.js** and import the **Navigation** component:

```
import { Navigation } from '@components/Navigation'
```

6. Then, define a sample **user** object to simulate a user being logged in:

```
export default function RootLayout({ children }) {  
  const user = { username: 'dan' }
```

7. Render the **Navigation** component, as follows:

```
return (  
  <html lang='en'>  
    <body>  
      <nav>  
        <Navigation username={user?.username} />  
      </nav>  
      <br />  
      <main>{children}</main>  
    </body>  
  </html>  
)  
}
```

8. We still need to add a link from a single post in the list to the full post page. Edit **src/components/Post.jsx** and import the **Link** component:

```
import Link from 'next/link'
```

9. Then, add a link to the title, as follows:

```
export function Post({ _id, title, author }) {  
  return (  
    <article>  
      <h3>
```

```
<Link href={` /posts/${_id}`}>{title}</Link>  
</h3>
```

10. Go to **http://localhost:3000/** and you will see the navigation bar being rendered with the **UserBar** component.
11. Click on the **Create Post** link to go to the corresponding page, then go back using the **Home** link. Also, try going to the full post page by clicking on the title of the blog post on the home page.

The following screenshot shows the **Home** page being rendered after we added the navigation bar:

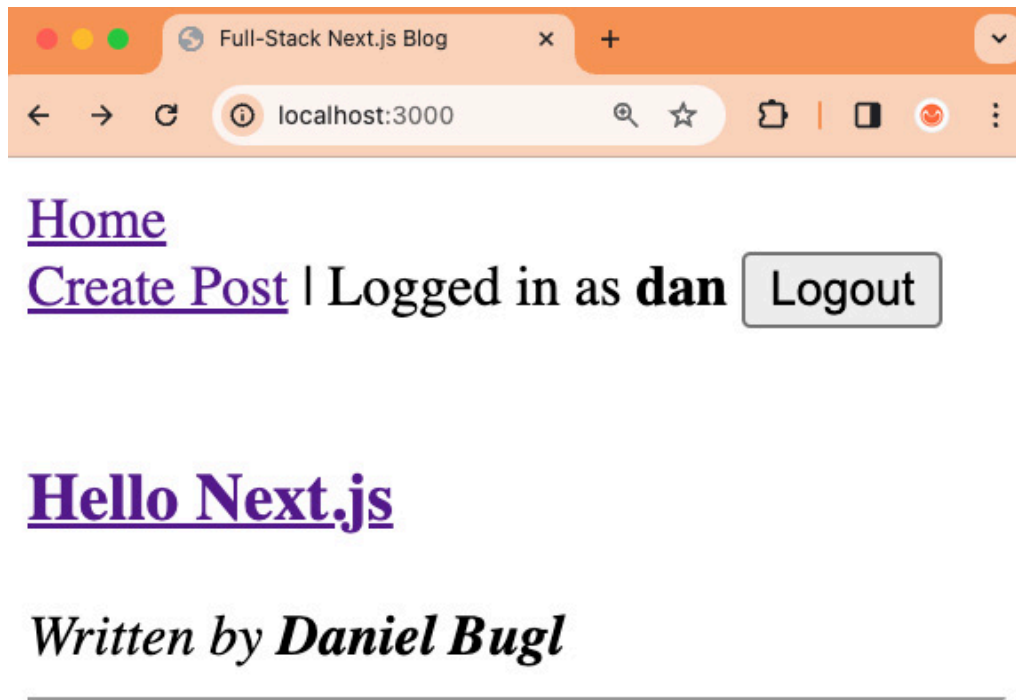


Figure 16.5 – Our (static) blog app re-created in Next.js!

Summary

In this chapter, we first learned what Next.js is and how it can be useful for full-stack development. Then, we set up a new Next.js project and learned about the App Router paradigm. Finally, we re-created the blog app in Next.js by creating components, pages, and a navigation bar, making use of the Next.js **Link** component to navigate between the different pages in our app.

In the next chapter, [*Chapter 17, Introducing React Server Components*](#), we are going to learn how to make our blog app interactive by creating React

Server Components, which are components that run on the server and can, for example, execute database queries. Additionally, we are going to learn about Server Actions, which are used to submit forms, such as the Login, Signup, and Create Post forms.